

Relatório – Implementação de Fila, Pilha e Deque em Sistema de Ingressos

Conceitos

Fila (FIFO – First In, First Out): A fila funciona de um jeito simples: quem entra primeiro é o primeiro a sair, igual a uma fila de pessoas esperando atendimento. No projeto, ela é usada pra organizar os ingressos comprados, garantindo que o público seja atendido na ordem certa.

Pilha (LIFO – Last In, First Out): Já a pilha é o contrário — o último item que entra é o primeiro que sai. Dá pra imaginar como uma pilha de pratos: o último que você coloca em cima é o primeiro que você pega. No sistema, ela serve pra guardar o histórico de navegação (como os botões “voltar” e “avançar”) e também pra funções de desfazer e refazer.

Deque (Double Ended Queue): O deque é uma estrutura parecida com a fila, mas mais versátil, porque dá pra adicionar e remover elementos tanto do começo quanto do fim. Essa flexibilidade faz dele uma ótima opção pra representar filas no sistema, já que permite gerenciar os atendimentos de forma rápida e eficiente.

Arquitetura e organização do código

Arquivos e papéis:

- fila.py → Implementa a estrutura de fila comum e a fila com prioridade (VIP, INTEIRA, MEIA).
- ingressos.py → Define os dados dos ingressos e as funções de estatísticas, salvar e carregar arquivos.
- pilha.py → Implementa as pilhas de navegação (voltar/avançar) e as de desfazer/refazer.
- roteiro.py → Gerencia as operações de navegação baseadas nas pilhas.
- main.py → Programa principal, responsável pela interação com o usuário via terminal, interpretando e executando os comandos.

Fluxo resumido do sistema:

1. O usuário inicia o programa (main.py), que cria o estado inicial do sistema.
2. Comandos são inseridos no terminal, como COMPRAR, ENTRAR, LISTAR, MODO, DESFAZER, entre outros.
3. Cada operação é registrada no histórico de ações e pode ser desfeita ou refeita por meio das pilhas.
4. As filas organizam os ingressos pendentes, e o modo de operação pode alternar entre PADRÃO e PRIORIDADE.
5. O estado pode ser salvo ou carregado em arquivos .json ou .csv.

Demonstrações com prints de tela

Figura 1 – Inicialização do sistema

```
PS C:\Users\João\Documents\python_trabalho> & C:/Python314/python.exe c:/Users/João/Documents/python_trabalho/terminal.py
> █
```

Figura 2 – Compra de ingressos

```
PS C:\Users\João\Documents\python_trabalho> & C:/Python314/python.exe c:/Users/João/Documents/python_trabalho/terminal.py
> Comprar Tulio VIP
OK: ingresso 1
> COMPRAR Bruno INTEIRA
OK: ingresso 2
> █
```

Legenda: Demonstra o enfileiramento de um ingresso na categoria VIP e Inteira.

Figura 3 – Alternância de modo

```
PS C:\Users\João\Documents\python_trabalho> & C:/Python314/python.exe c:/Users/João/Documents/python_trabalho/terminal.py
> Comprar Tulio VIP
OK: ingresso 1
> COMPRAR Bruno INTEIRA
OK: ingresso 2
> Modo Prioridade
OK
> █
```

Legenda: Indica a troca do modo padrão para o modo de prioridade, redistribuindo os ingressos

entre as filas.

Figura 4 – Entrada de cliente

```
PS C:\Users\João\Documents\python_trabalho> & C:/Python314/python.exe c:/Users/João/Documents/python_trabalho/terminal.py
> Comprar Tulio VIP
OK: ingresso 1
> COMPRAR Bruno INTEIRA
OK: ingresso 2
> Modo Prioridade
OK
> ENTRAR
Entrada: [1] Tulio (VIP)
> █
```

Legenda: Mostra a remoção (desenfileiramento) do ingresso na ordem de prioridade.

Figura 5 – Cancelamento e listagem

```
OK: ingresso 2
> Modo Prioridade
OK
> ENTRAR
Entrada: [1] Tulio (VIP)
> CANCELAR 1
Erro: Ingresso ja atendido
> CANCELAR 2
OK: ingresso 2 cancelado
> Listar
Nenhum pendente
> █
```

Legenda: Comprova a remoção de um ingresso pendente e a atualização da fila.

Figura 6 – Estatísticas

```
OK
> ENTRAR
Entrada: [1] Tulio (VIP)
> CANCELAR 1
Erro: Ingresso ja atendido
> CANCELAR 2
OK: ingresso 2 cancelado
> Listar
Nenhum pendente
> ESTATISTICAS
pendentes=0, atendidos=1, por_categoria={'VIP': 1, 'INTEIRA': 0, 'MEIA': 0}, espera_media=0.0
> █
```

Legenda: Demonstra a análise dos dados armazenados.

Figura 7 – Navegação entre páginas

```
OK
> ENTRAR
Entrada: [1] Tulio (VIP)
> Ir /Palco
OK: /Palco
> Ir /compras
OK: /compras
> Voltar
OK: /Palco
> ONDE
/Palco
> █
```

Legenda: Ilustra o funcionamento do roteiro com histórico de navegação baseado em pilhas.

Figura 8 – Desfazer e refazer

```
OK: /compras
> Voltar
OK: /Palco
> ONDE
/Palco
> Desfazer
OK: desfaz VOLTAR
> refazer
OK
> onde
/Palco
>
```

Legenda: Comprova o controle de histórico de ações por meio das pilhas de undo/redo.

Figura 9 – Mensagens de erro

```
PS C:\Users\João\Documents\python_trabalho> & C:/Python314/python.exe c:/Users/João/Documents/python_trabalho/terminal.py
> comprar
Uso: COMPRAR <nome> <categoria>
> entrar
Fila vazia
> comprar Bruno
Uso: COMPRAR <nome> <categoria>
> Cancelar
Uso: CANCELAR <id>
> Modo
Uso: MODO PADRAO|PRIORIDADE
> Ir
Uso: IR <caminho>
> Salvar
Uso: SALVAR <arquivo>
> Carregar
Uso: CARREGAR <arquivo>
> COMPRAR Tullio COMPLETO
Erro: Categoria invalida
> ESPIAR
Fila vazia
> COMPRAR Bruno INTEIRA
OK: ingresso 1
> CANCELAR 999
Erro: Ingresso inexistente
> ENTRAR
Entrada: [1] Bruno (INTEIRA)
> CANCELAR 1
Erro: Ingresso ja atendido
```

Legenda: Mostra o tratamento de exceções e validações de entrada.

Figura 10 – Sair

```
> Comprar Tullio vip
OK: ingresso 1
> comprar joao meia
OK: ingresso 2
> ajuda
Comandos: COMPRAR/ENTRAR/ESPIAR/CANCELAR/LISTAR/ESTATISTICAS/MODO/IR/VOLTAR/AVANCAR/ONDE/MAPA/DESAZER/REFAZER/SALVAR/CARREGAR/AJUDA/SAIR
> sair
PS C:\Users\João\Documents\python_trabalho>
```

Legenda: Comprova que o programa desliga corretamente ao sair.

Conclusão

O projeto implementa de forma completa três conceitos fundamentais de estruturas de dados — filas, pilhas e deque — aplicados em um sistema prático de controle de ingressos. Foram desenvolvidas as seguintes funcionalidades:

- Fila padrão e fila com prioridade, utilizando deque para eficiência;
- Roteiro de navegação com voltar e avançar, baseado em pilhas LIFO;
- Mecanismos de desfazer e refazer, permitindo reverter operações anteriores;
- Persistência de dados por meio das funções de salvar e carregar estados;
- Validações e mensagens de erro, garantindo robustez no uso via terminal.

O sistema demonstra de forma integrada como estruturas de dados clássicas podem ser aplicadas a contextos reais, reforçando os conceitos de organização lógica e modularidade no desenvolvimento de software.

Dificuldades e melhorias

A maior dificuldade que o grupo enfrentou no desenvolvimento do programa foi implementar a compra de diferentes tipos de ingressos e, depois, colocar o comprador na fila. No início, o programa aceitava a compra, mas não registrava o comprador. Depois conseguimos fazer o registro, mas ainda sem exibir o nome de quem estava na fila ou de quem já havia entrado no evento. Por isso, acabamos gastando um bom tempo até tudo funcionar corretamente, com a saída de acordo com as instruções. Como melhoria, a única implementação que consideramos viável seria permitir a troca dos ingressos após a compra ou o reembolso deles.